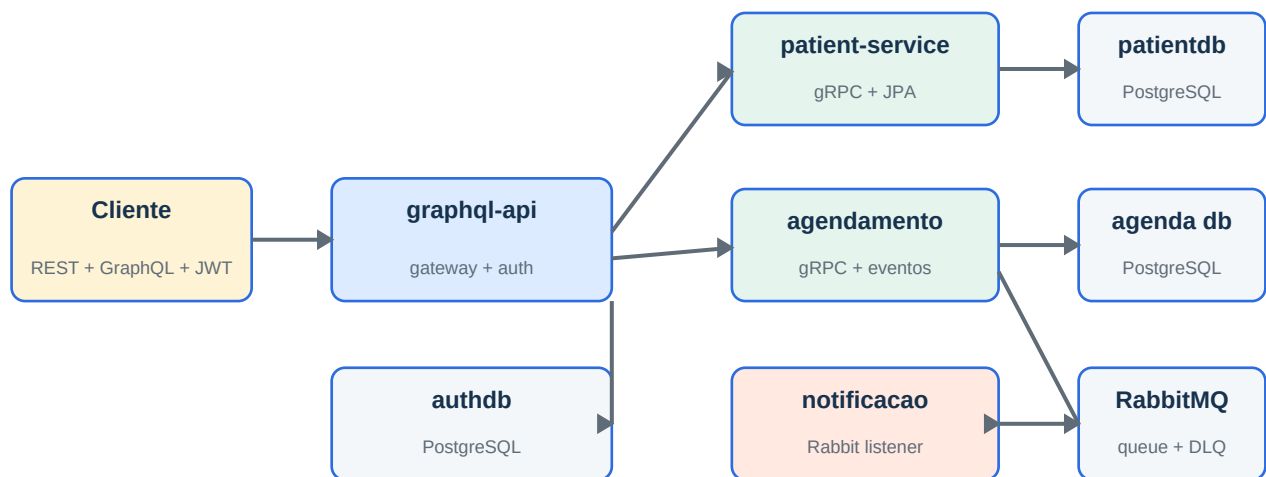


CARESYNC

Arquitetura, casos de uso, execução e testes



Tech Challenge - Fase 3

Versão técnica consolidada - Setembro de 2026

1. Visão geral

CareSync é um backend hospitalar composto por quatro serviços independentes. O sistema protege operações por perfil, oferece consultas flexíveis via GraphQL, usa gRPC na comunicação síncrona interna e RabbitMQ para notificação assíncrona.

Objetivos atendidos

- Autenticação JWT implementada com Spring Security e senhas BCrypt.
- Médicos visualizam históricos e editam consultas; enfermeiros visualizam e agendam; pacientes acessam apenas seus próprios dados.
- Histórico completo e consultas futuras disponíveis via GraphQL.
- Eventos publicados em criação e edição, com retry e dead-letter queue.
- PostgreSQL isolado por serviço persistente e migrações Flyway.
- Gate JaCoCo de 95% para linhas e branches em cada módulo.

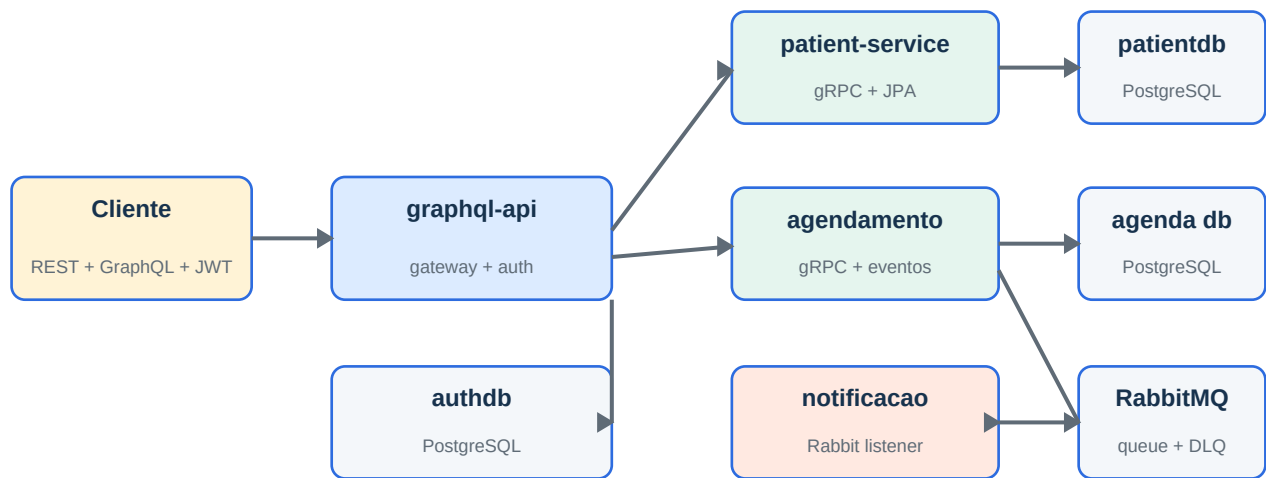
Matriz de componentes

Componente	Responsabilidade	Tecnologia	Dados
graphql-api	Gateway, segurança e usuários	REST, GraphQL, JWT, gRPC	authdb
patient-service	Localizar pacientes	gRPC, JPA	patientdb
agendamento-service	Criar, editar e listar consultas	gRPC, JPA, AMQP	agendamentodb
notificacao-service	Consumir eventos e enviar lembrete	RabbitMQ, log estruturado	Stateless

Decisão importante

O envio externo foi modelado como porta. Nesta versão, o adaptador grava um log estruturado; SMTP, SMS ou push podem ser adicionados sem alterar o caso de uso.

2. Arquitetura geral



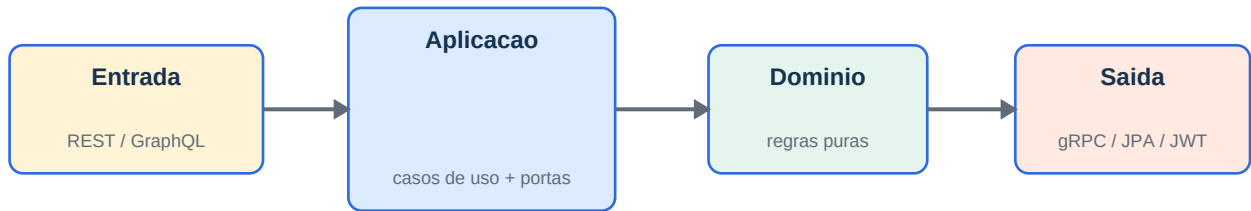
Princípios

- Database per service: nenhum serviço consulta diretamente o banco de outro.
- Dependências apontam para dentro: adaptadores dependem de portas; domínio não conhece frameworks.
- graphql-api é a única entrada pública funcional.
- Contratos gRPC são definidos por Protobuf e gerados separadamente no cliente e servidor.
- Mensageria desacopla agendamento e notificação; falhas permanentes seguem para notificacao.dlq.

Fluxo síncrono e assíncrono

Etapas	Origem	Destino	Resultado
1	Cliente	graphql-api	JWT autenticado e role autorizada
2	graphql-api	patient-service	Paciente validado por gRPC
3	graphql-api	agendamento-service	Consulta persistida por gRPC
4	agendamento-service	RabbitMQ	ConsultaEvent publicado
5	RabbitMQ	notificacao-service	Lembrete processado ou enviado à DLQ

3. graphql-api



Casos de uso

- Autenticar usuário e emitir JWT.
- Criar, listar e remover usuários administrativos.
- Orquestrar paciente e consultas por gRPC.
- Aplicar autorização por perfil e propriedade do paciente.

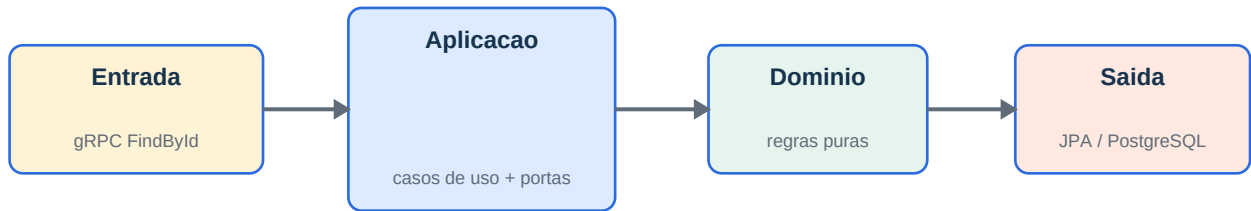
Contratos e interfaces

Interface	Descrição
POST /auth/login	Contrato ativo
POST, GET e DELETE /users	Contrato ativo
POST /graphql	Contrato ativo
GET /graphql e /swagger-ui/index.html	Contrato ativo

Dados e operação

PostgreSQL authdb. Flyway cria app_users; a carga idempotente gera cinco contas acadêmicas com BCrypt.

4. patient-service



Casos de uso

- Validar identificador positivo.
- Localizar o paciente por sua porta de repositório.
- Retornar NOT_FOUND sem expor detalhes internos.
- Mapear domínio para o contrato Protobuf.

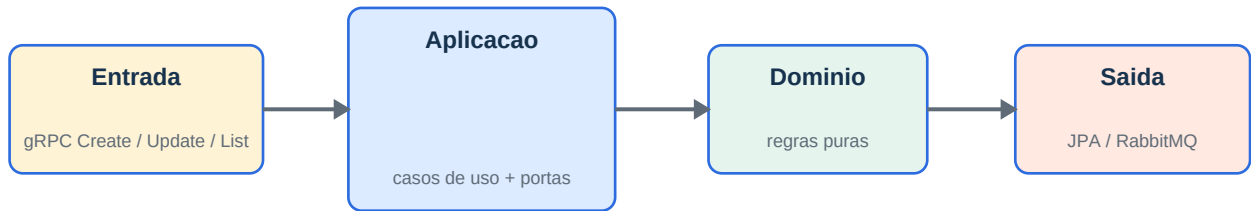
Contratos e interfaces

Interface	Descrição
PatientService.FindById(PatientRequest)	Contrato ativo
PatientResponse: id, name, email	Contrato ativo

Dados e operação

PostgreSQL patientdb. Flyway cria patients e carrega Maria Silva e Joao Souza com IDs determinísticos.

5. agendamento-service



Casos de uso

- Agendar somente no futuro e com campos obrigatórios.
- Editar consulta existente e validar status.
- Listar histórico ordenado e filtrar somente futuras.
- Publicar CREATED ou UPDATED após persistência.

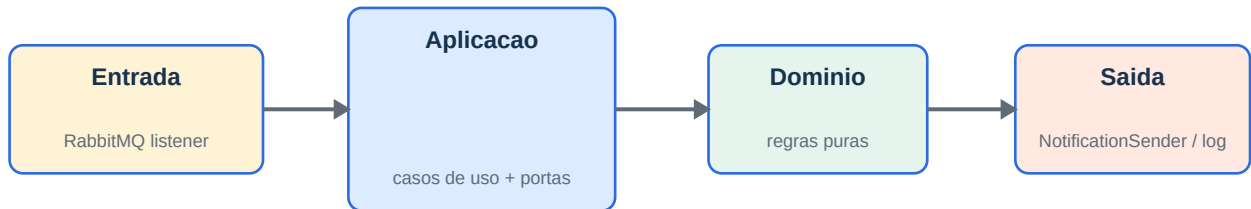
Contratos e interfaces

Interface	Descrição
Create	Contrato ativo
Update	Contrato ativo
ListByPatient	Contrato ativo
ListUpcomingByPatient	Contrato ativo

Dados e operação

PostgreSQL agendamentodb. Status: SCHEDULED, COMPLETED e CANCELLED. Índice por patient_id e date_time.

6. notificacao-service



Casos de uso

- Consumir ConsultaEvent.
- Aceitar CREATED e UPDATED.
- Construir uma notificação de domínio validada.
- Repetir até três vezes e rotear falhas permanentes à DLQ.

Contratos e interfaces

Interface	Descrição
Exchange consulta.exchange	Contrato ativo
Fila notificacao.queue	Contrato ativo
DLQ notificacao.dlq	Contrato ativo
Binding consulta.*	Contrato ativo

Dados e operação

Serviço stateless. Não possui banco por decisão arquitetural; duplicidade deve ser tratada pelo futuro adaptador externo quando necessário.

7. Segurança e API

O login público autentica username e senha. O token assinado contém o subject, emissão e expiração de uma hora. Requisições sem token, com token inválido ou expirado recebem HTTP 401.

Operação	ADMIN	MÉDICO	ENFERMEIRO	PACIENTE
Gerenciar usuários	Sim	Não	Não	Não
Consultar paciente/histórico	Não	Qualquer	Qualquer	Próprio
Agendar consulta	Não	Não	Sim	Não
Editar consulta	Não	Sim	Não	Não

Exemplos

POST /auth/login

```
{"username": "medico1", "password": "senha123"}
```

GraphQL - Authorization: Bearer <token>

```
query { consultasFuturasDoPaciente(patientId: 1) { id dateTime status } }
```

Erros

- BAD_REQUEST: entrada ou regra de negócio inválida.
- FORBIDDEN: role sem permissão ou paciente tentando acessar outro cadastro.
- NOT_FOUND: consulta ou paciente inexistente.
- INTERNAL_ERROR: dependência indisponível sem vazamento de stack trace.

8. Bancos e mensageria

O Docker Compose sobe três instâncias PostgreSQL independentes. Cada aplicação valida o schema após o Flyway executar; Hibernate não cria nem altera tabelas.

Banco	Porta local	Tabela	Massa
patientdb	5432	patients	2 pacientes
agendamentodb	5433	consultas	1 passada e 2 futuras
authdb	5434	app_users	5 perfis

Evento ConsultaEvent

consultaId, patientId, doctorName, dateTime, reason e eventType. O produtor usa routing key consulta.created ou consulta.updated. O consumidor rejeita tipos desconhecidos, ativa retry e encaminha a mensagem à notificacao.dlq após esgotar três tentativas.

Consistência

A consulta é salva antes da publicação. Se o broker estiver indisponível, a chamada falha de forma observável; para garantias de entrega exatamente após commit em um ambiente produtivo, recomenda-se evoluir para transactional outbox.

9. Como executar

Pré-requisito: Docker Desktop ou Docker Engine com Compose em execução.

```
docker compose up --build
```

Endereços

Recurso	Endereço
GraphQL	http://localhost:8081/graphql
GraphiQL	http://localhost:8081/graphiql
Swagger	http://localhost:8081/swagger-ui/index.html
RabbitMQ	http://localhost:15672
gRPC pacientes	localhost:9090
gRPC consultas	localhost:9091

Configuração

- PATIENT_DB_URL, PATIENT_DB_USER, PATIENT_DB_PASSWORD.
- AGENDAMENTO_DB_URL, AGENDAMENTO_DB_USER, AGENDAMENTO_DB_PASSWORD.
- AUTH_DB_URL, AUTH_DB_USER, AUTH_DB_PASSWORD.
- RABBITMQ_HOST, PATIENT_SERVICE_HOST, AGENDAMENTO_SERVICE_HOST e JWT_SECRET.

Reinício limpo

```
docker compose down -v  
docker compose up --build
```

10. Como testar

Cada módulo executa testes e o gate JaCoCo no lifecycle verify. O build falha se linhas ou branches ficarem abaixo de 95%. Somente fontes Java geradas pelo Protobuf são excluídas.

```
cd patient-service && ./mvnw clean verify
cd agendamento-service && ./mvnw clean verify
cd notificacao-service && ./mvnw clean verify
cd graphql-api && ./mvnw clean verify
```

Cobertura validada

Módulo	Linhas	Branches	Gate
patient-service	97.22%	100%	Aprovado
agendamento-service	98.95%	100%	Aprovado
notificacao-service	97.14%	100%	Aprovado
graphql-api	99.55%	100%	Aprovado

Aceitação funcional

```
npx newman run CareSync.postman_collection.json
```

- Login válido para cinco perfis e login inválido.
- Criação, listagem, autorização e limpeza de usuário.
- Acesso próprio do paciente e bloqueio de acesso cruzado.
- Histórico completo e filtro de futuras.
- Agendamento por enfermeiro e edição por médico com ID dinâmico.
- Negativas por role, data inválida, paciente inexistente, ausência e corrupção de token.

11. Operação e troubleshooting

Sintoma	Verificação	Ação
Docker não conecta	Docker Engine/pipe indisponível	Iniciar Docker Desktop e repetir compose up
Migration falha	Logs Flyway e credenciais	Revisar URL/usuário; em dev, recriar volumes
UNAVAILABLE no GraphQL	Serviços gRPC e portas 9090/9091	Aguardar inicialização e conferir hosts
Evento na DLQ	RabbitMQ Management e logs	Corrigir payload/adaptador e reprocesar conscientemente
401	Header e expiração JWT	Autenticar novamente e usar Bearer token

Checklist de entrega

- Quatro módulos compilam e executam testes.
- Gates JaCoCo aprovados para linhas e branches.
- Collection Postman é JSON válido e executável em sequência.
- Docker Compose é sintaticamente válido e declara healthchecks.
- Schemas e massas são determinísticos e idempotentes.
- README contém diagramas Mermaid, badges, contratos e instruções.
- PDF foi extraído, renderizado e inspecionado página a página.

Fim da documentação